Clase 4 – Algoritmos básicos y arreglos (Versión Completa)

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano

Contenidos

- Punteros y sus operadores
- Vectores y matrices
- Recorridos con `for`
- Búsqueda lineal
- Ordenamiento burbuja básico

Objetivos

Herramientas

OnlineGDB es un entorno de desarrollo en la nube (IDE) que permite editar, compilar y ejecutar programas en **C/C++** con un solo clic:

https://www.onlinegdb.com/

Repaso de la unidad anterior

- **Constantes:** `const` y `#define`
- **Variables:** cómo declararlas
- **Tipos de datos:** `char`, `int`, `float`, `double`
- **Estructuras de control:**
  - *Selectivas:* `if`, `case`
  - *Repetitivas:* `for`, `while`, `do while`

Anatomía básica de una computadora

Componente	Descripción
CPU	Ejecuta las instrucciones de los programas.
Memoria RAM	Almacena datos e instrucciones en uso. Es volátil.
Almacenamiento (Disco/SSD)	Guarda el sistema operativo, programas y archivos.
Periféricos y microcontroladores	Gestionan entrada/salida (teclado, sensores, puertos).

Componente	Descripción
🔄 **Buses del sistema**	Conectan los componentes y permiten la comunicación.

💡 La memoria RAM

Podemos imaginar la **RAM** como una tabla con filas numeradas:

```
+-------------+-------------+
| Dirección   | Contenido   |
+-------------+-------------+
| 1000        |   5         |
| 1004        |   9         |
| 1008        |  'A'        |
+-------------+-------------+
```

- Cada fila tiene una **dirección** (número identificador).
- Cada dirección tiene un **contenido** (valor guardado).
- Una variable ocupa un casillero.
- El **nombre** de la variable es un "alias" para esa dirección.

🚀 El viaje de un programa

¿Qué pasa cuando ejecutamos un programa?

1. El sistema operativo carga el programa en memoria.
2. La memoria se divide en secciones:
    - **Código:** instrucciones del programa
    - **Datos:** variables globales y estáticas
    - **Pila (Stack):** variables locales y control de funciones
    - **Heap:** memoria dinámica bajo demanda
3. La CPU ejecuta las instrucciones en ciclos.

```
┌──────────────────────────────────┐
│ Memoria RAM                      │
│ ┌──────────────┐                 │
│ │ Código       │ → Instrucciones del programa │
│ ├──────────────┤                 │
│ │ Datos        │ → Variables globales         │
│ ├──────────────┤                 │
│ │ Pila (Stack)│ → Variables locales, control │
│ ├──────────────┤                 │
│ │ Heap         │ → Memoria dinámica (malloc)  │
└──────────────────────────────────┘
```

¿Qué es un puntero?

Un **puntero** es una variable especial que guarda **direcciones de memoria**.

Se dice que "apunta" a una dirección que contiene un valor de un tipo específico.

Definición

```
tipo *nombrePuntero;
```

Operadores

- **& (Referencia):** obtiene la dirección de memoria.
- *** (Desreferencia):** accede al valor guardado en esa dirección.

```c
#include <stdio.h>

int main() {
    int x = 5;          // Variable normal
    int *px = &x;       // Puntero que guarda la dirección de x

    printf("Valor de x: %d\n", x);
    printf("Dirección de x: %p\n", &x);
    printf("Valor de px (direccion que apunta): %p\n", px);
    printf("Valor al que apunta px: %d\n", *px);
    return 0;
}
```

Diagrama ASCII

```
+-----------+-----------+
| Nombre    | Dirección | Valor |
+-----------+-----------+-------+
| x         | 1000      |   5   |
| px        | 1004      | 1000  |
+-----------+-----------+-------+
```

Vectores

- Una **variable** ocupa un casillero en memoria.

```c
int x = 5;
```

- Un **puntero** guarda una dirección de memoria.

```
int *px = &x;
```

- Un **vector** ocupa varios casilleros contiguos.

```
int vector[10];
```

  - El índice va de `0` a `n-1`.

```c
#include <stdio.h>

int main() {
    int vector[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        printf("Elemento %d = %d\n", i, vector[i]);
    }
    return 0;
}
```

Representación en memoria

```
Dirección: 1000 1004 1008 1012 1016
Contenido:  10   20   30   40   50
             ↑    ↑    ↑    ↑    ↑
           v[0] v[1] v[2] v[3] v[4]
```

🔢 Arreglos (Matrices)

Un **arreglo bidimensional** se comporta como una tabla:

Tipo de arreglo	Ejemplo	Descripción
Unidimensional	`int vector[5];`	Vector
Bidimensional	`int matriz[3][5];`	Matriz
Multidimensional	`int tridimensional[4][4][4];`	Más de dos dimensiones

```c
#include <stdio.h>

int main() {
    int matriz[2][3] = {
```

```
        {1, 2, 3},
        {4, 5, 6}
    };
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matriz[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Representación en memoria (2x3):

```
       Columnas →
        0   1   2
Fila 0 [1] [2] [3]
Fila 1 [4] [5] [6]
```

Recorridos con for

Recorrido de un vector

```
for (int i = 0; i < n; i++) {
    printf("%d ", vector[i]);
}
```

Recorrido de una matriz

```
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        printf("%d ", matriz[i][j]);
    }
}
```

Operaciones con vectores

◇ Ordenamiento burbuja (Bubble Sort)

Método simple para ordenar elementos:

1. Compara elementos adyacentes.
2. Los intercambia si están en orden incorrecto.

3. Repite el proceso hasta ordenar el vector.
4. En cada pasada, el mayor “burbujea” al final.

```c
#include <stdio.h>

int main() {
    int vector[] = {5, 2, 9, 1, 5, 6};
    int n = 6;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (vector[j] > vector[j + 1]) {
                int temp = vector[j];
                vector[j] = vector[j + 1];
                vector[j + 1] = temp;
            }
        }
    }

    printf("Vector ordenado:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", vector[i]);
    }
    return 0;
}
```

Representación del proceso:

```
[5, 2, 9, 1, 5, 6]
→ [2, 5, 1, 5, 6, 9]
→ [2, 1, 5, 5, 6, 9]
→ [1, 2, 5, 5, 6, 9]
```

Cada pasada “burbujea” el mayor al final.

Búsqueda lineal en un vector

Búsqueda Lineal en un Vector

- La búsqueda lineal consiste en revisar cada elemento del vector de manera secuencial hasta encontrar el valor buscado.
- Es simple y no requiere que el vector esté ordenado, aunque funciona igual en vectores ordenados.

```c
#include <stdio.h>

int main() {
    int vector[] = {4, 7, 1, 9, 3};
```

```
    int valorBuscado = 9;
    int n = 5;

    for (int i = 0; i < n; i++) {
        if (vector[i] == valorBuscado) {
            printf("Encontrado en la posición %d\n", i);
            return 0;
        }
    }
    printf("Valor no encontrado\n");
    return 0;
}
```

🔎 Visualización

```
vector = [4, 7, 1, 9, 3]
            ↑
Comparaciones → 4 → 7 → 1 → ☑ 9 (encontrado en posición 3)
```

📚 Bibliografía

- *El lenguaje C: Manual de Referencia* — Balestrini & Di Stefano
- *Programación en C para principiantes* — Urrutia & Landa
- *Introducción a la programación en C* — Marzal & Gracia
- *GNU C Language Introduction and Reference Manual* — Richard Stallman

---